Memory Management

(Chaper 4, Tanenbaum)

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Introduction

The CPU fetches instructions and data of a program from memory; therefore, both the program and its data must reside in the main (RAM and ROM) memory.

Modern multiprogramming systems are capable of storing more than one program, together with the data they access, in the main memory.

A fundamental task of the **memory management** component of an operating system is to ensure safe execution of programs by providing:

- Sharing of memory
- Memory protection

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Issues in sharing memory

- *Transparency*

  Several processes may co-exist, unaware of each other, in the main memory and run regardless of the number and location of processes.

- *Safety (or protection)*

  Processes must not corrupt each other (nor the OS!)

- *Efficiency*

  CPU utilization must be preserved and memory must be fairly allocated. Want low overheads for memory management.

- *Relocation*

  Ability of a program to run in different memory locations.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Storage allocation

Information stored in main memory can be classified in a variety of ways:

- Program (*code*) and data (*variables, constants*)
- Read-only (*code, constants*) and read-write (*variables*)
- Address (*e.g., pointers*) or data (*other variables*); binding (when memory is allocated for the object): static or dynamic

The compiler, linker, loader and run-time libraries all cooperate to manage this information.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

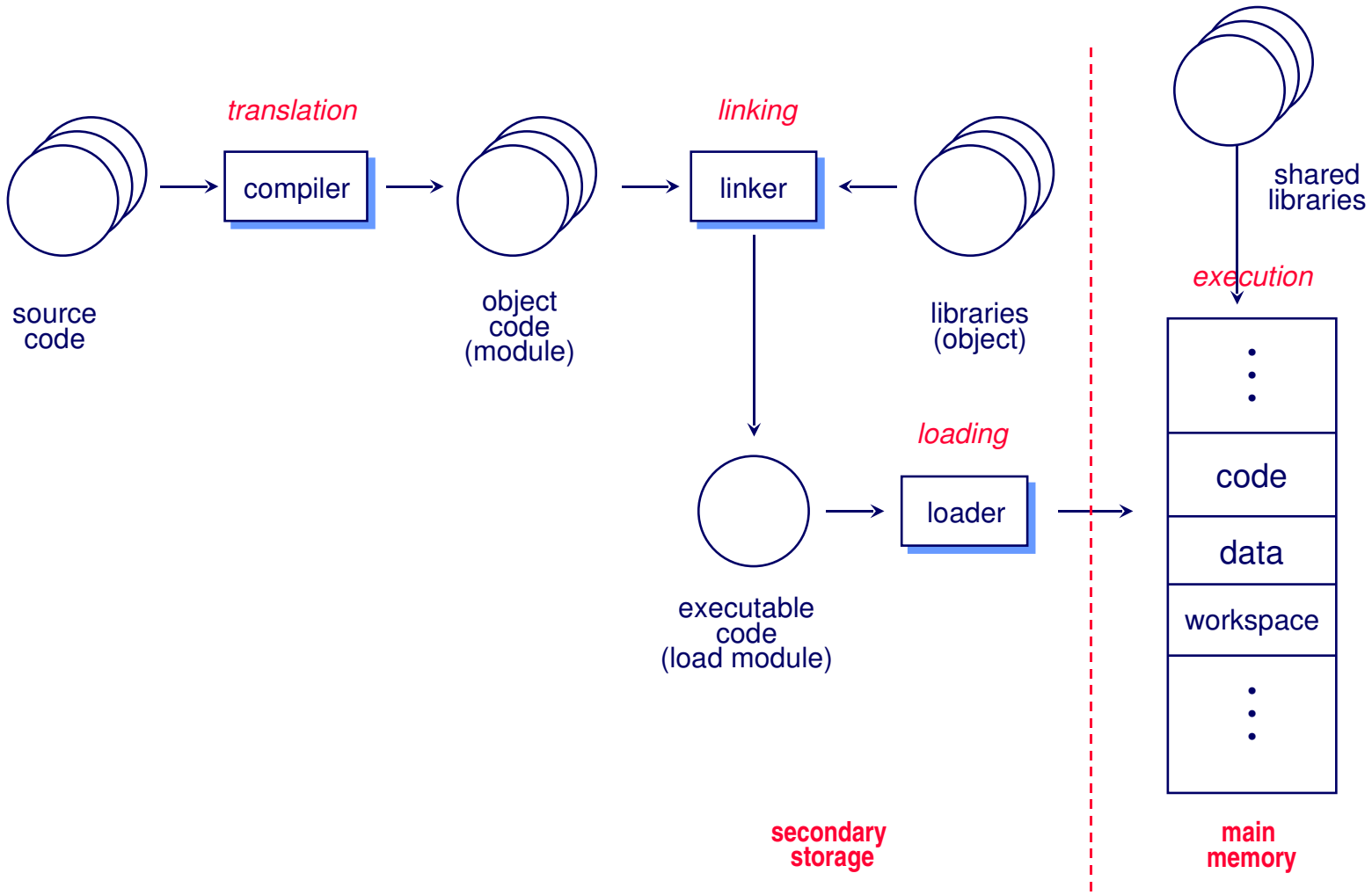
Creating an executable code

Before a program can be executed by the CPU, it must go through several steps:

- **Compiling (translating)**—generates the object code.
- **Linking**—combines the object code into a single self-sufficient *executable code*.
- **Loading**—copies the executable code into memory. May include run-time linking with libraries.
- **Execution**—dynamic memory allocation.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

From *source* to *executable* code



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Address binding (relocation)

The process of associating program instructions and data (addresses) to physical memory addresses is called *address binding*, or *relocation*.

- **Static**—new locations are determined *before* execution.
  - Compile time: The compiler or assembler translates *symbolic* addresses (e.g., variables) to *absolute* addresses.
  - Load time: The compiler translates symbolic addresses to *relative* (*relocatable*) addresses. The loader translates these to absolute addresses.
- **Dynamic**—new locations are determined *during* execution.
  - Run time: The program retains its relative addresses. The absolute addresses are generated by hardware.

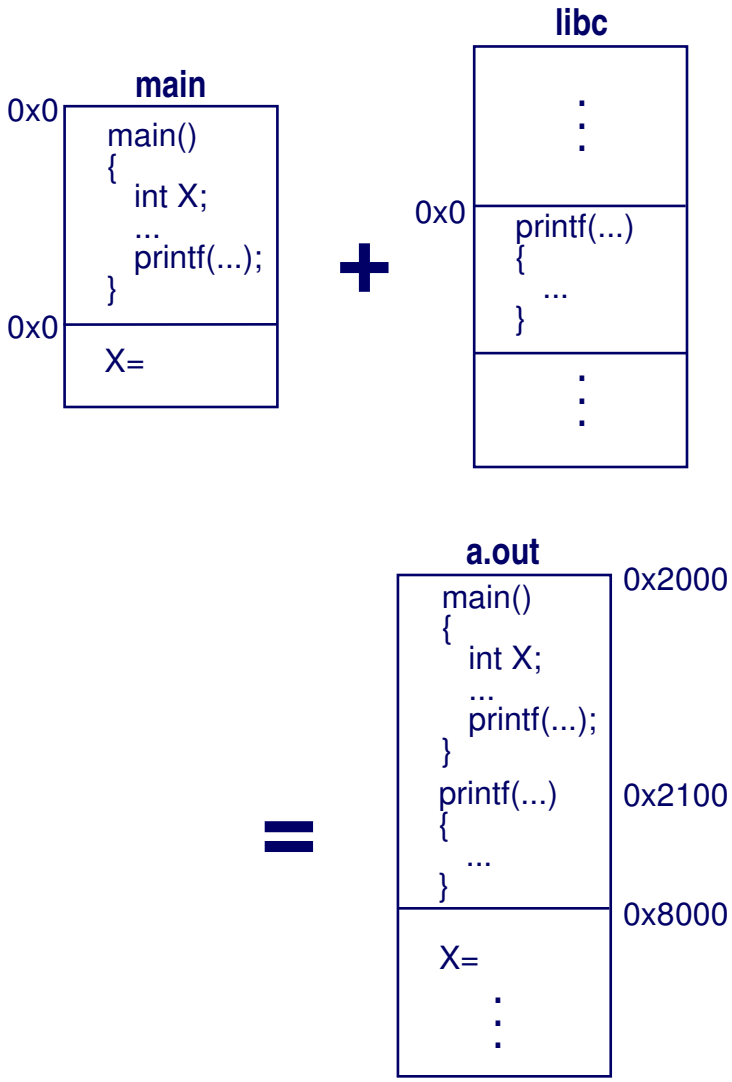
Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Functions of a linker

A compile-time *linker* collects (if possible) and puts together all the required pieces to form the executable code.

Issues:

- **Relocation**
  where to *put* pieces.
- **Cross-reference**
  where to *find* pieces.
- **Re-organization**
  new memory layout.

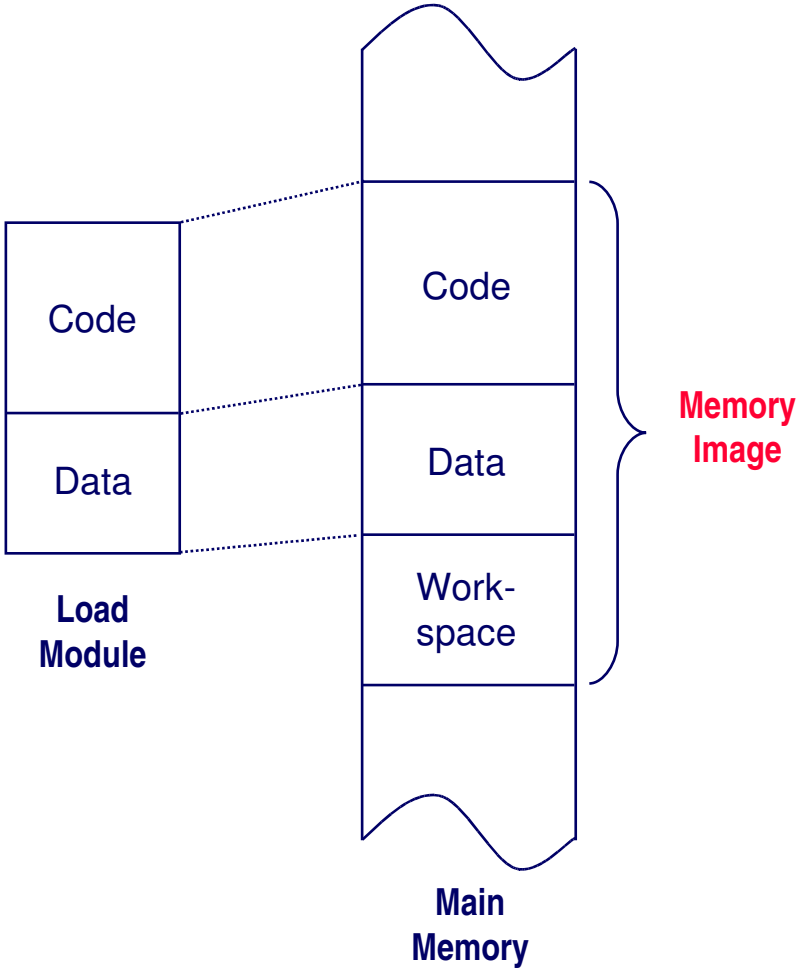


Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Functions of a loader

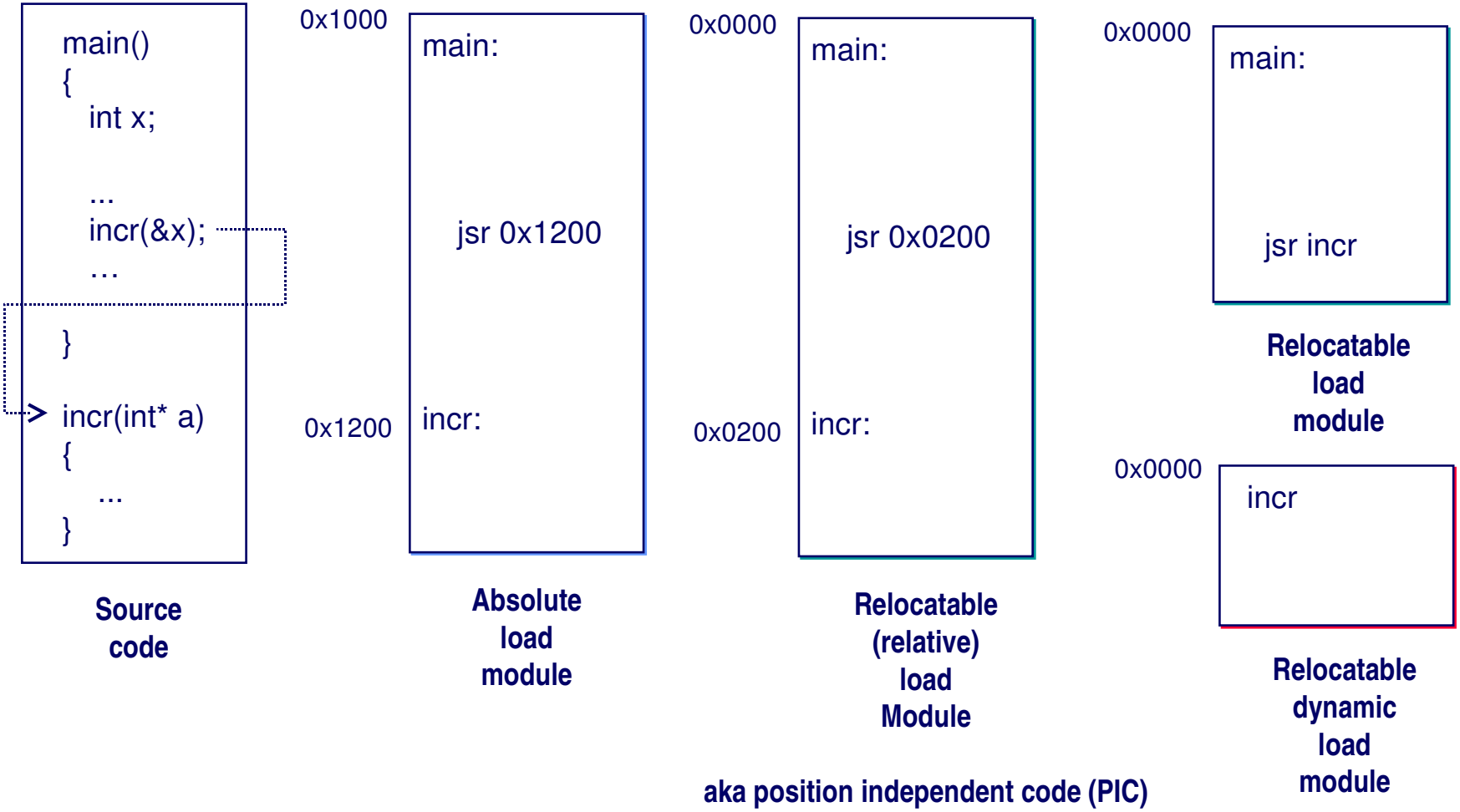
A *loader* places the executable code in main memory starting at a pre-determined location (base or start address). This can be done in several ways, depending on hardware architecture:

- *Absolute loading:* always loads programs into a designated memory location.
- *Relocatable loading:* allows loading programs in different memory locations.
- *Dynamic (run-time) loading:* loads functions when first called (if ever).



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Absolute and relocatable modules



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Simple management schemes

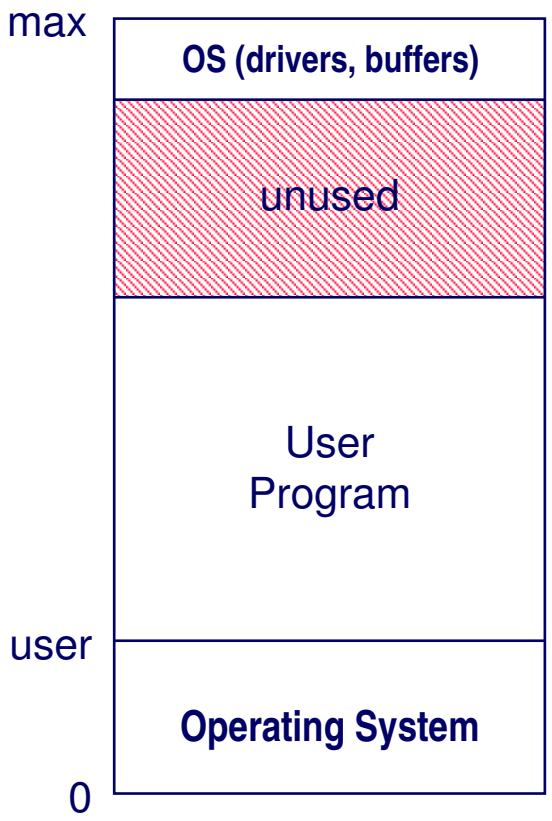
An important task of a memory management system is to bring (load) programs into main memory for execution. The following *contiguous memory allocation* techniques were commonly employed by earlier operating systems*:

- Direct placement
- Overlays
- Partitioning

*Note: Techniques similar to those listed above are still used by some modern, dedicated special-purpose operating systems and real-time systems.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Direct placement



Memory allocation is trivial. No special relocation is needed, because the user programs are always loaded (one at a time) into the same memory location (*absolute loading*). The linker produces the same loading address for every user program.

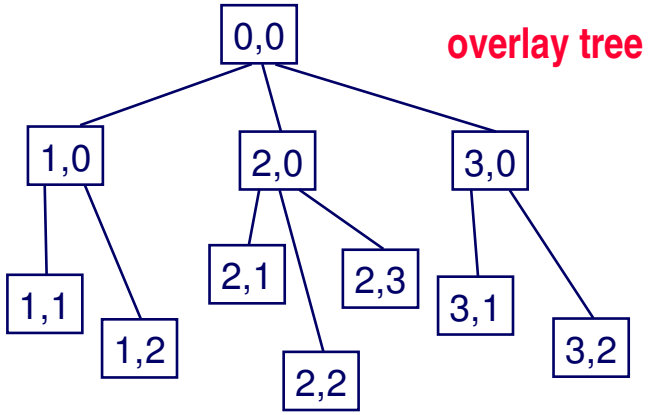
Examples:

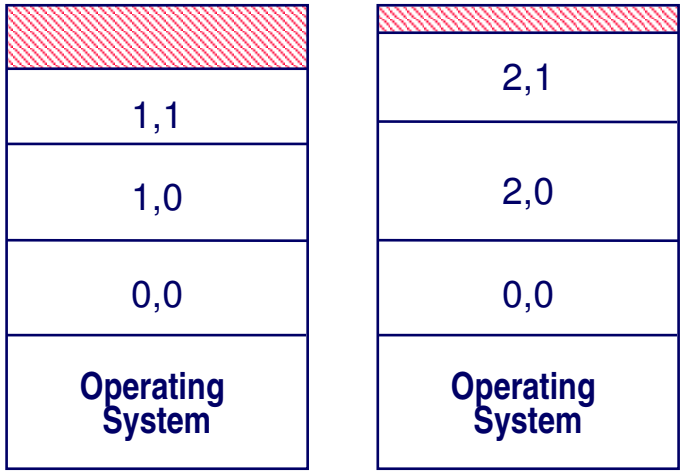
- Early batch monitors
- MS-DOS

July 99

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Overlays





memory snapshots

Historically, before the sharing of main memory among several programs was automated, users developed techniques to allow large programs to execute (fit) in smaller memory.

A program was organized (by the user) into a tree-like structure of object modules, called *overlays*.

The *root* overlay was always loaded into the memory, whereas the sub-tree were (re-)loaded as needed by simply overlaying existing code.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Partitioning

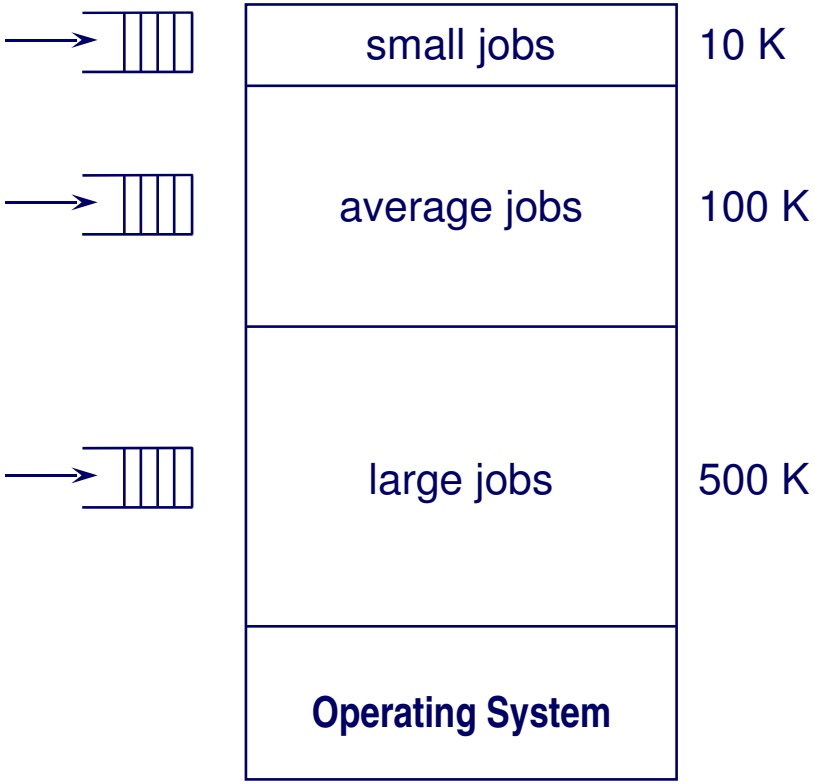
A simple method to accommodate several programs in memory at the same time (to support multiprogramming) is partitioning. In this scheme, the memory is divided into a number of contiguous regions, called *partitions*. Two forms of memory partitioning, depending on when and how partitions are created (and modified), are possible:

- Static partitioning
- Dynamic partitioning

These techniques were used by the IBM OS/360 operating system—*MFT* (*M*ultiprogramming with *F*ixed Number of *T*asks) and *MVT* (*M*ultiprogramming with *V*ariable Number of *T*asks.)

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Static partitioning



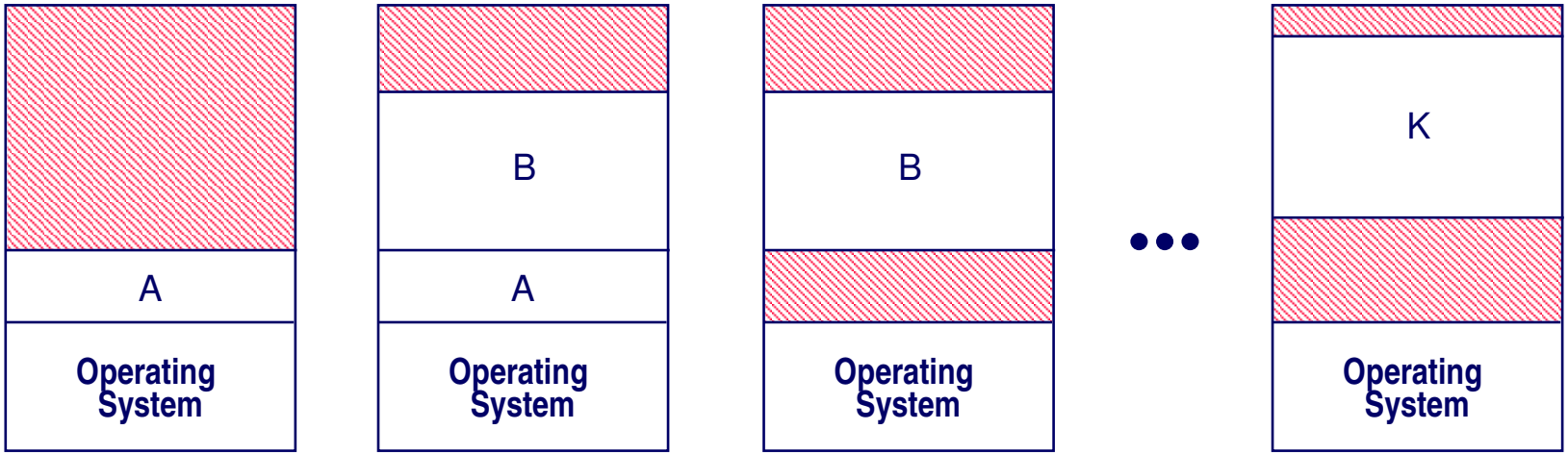
Main memory is divided into *fixed* number of (fixed size) partitions during system generation or startup.

Programs are queued to run in the smallest available partition. An executable prepared to run in one partition may not be able to run in another, <u>without being re-linked</u>. This technique uses *absolute loading*.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Dynamic partitioning

Any number of programs can be loaded to memory as long as there is room for each. When a program is loaded (*relocatable loading*), it is allocated memory in exact amount as it needs. Also, the addresses in the program are fixed after loaded, so it cannot move. The operating system keeps track of each partition (their size and locations in the memory.)



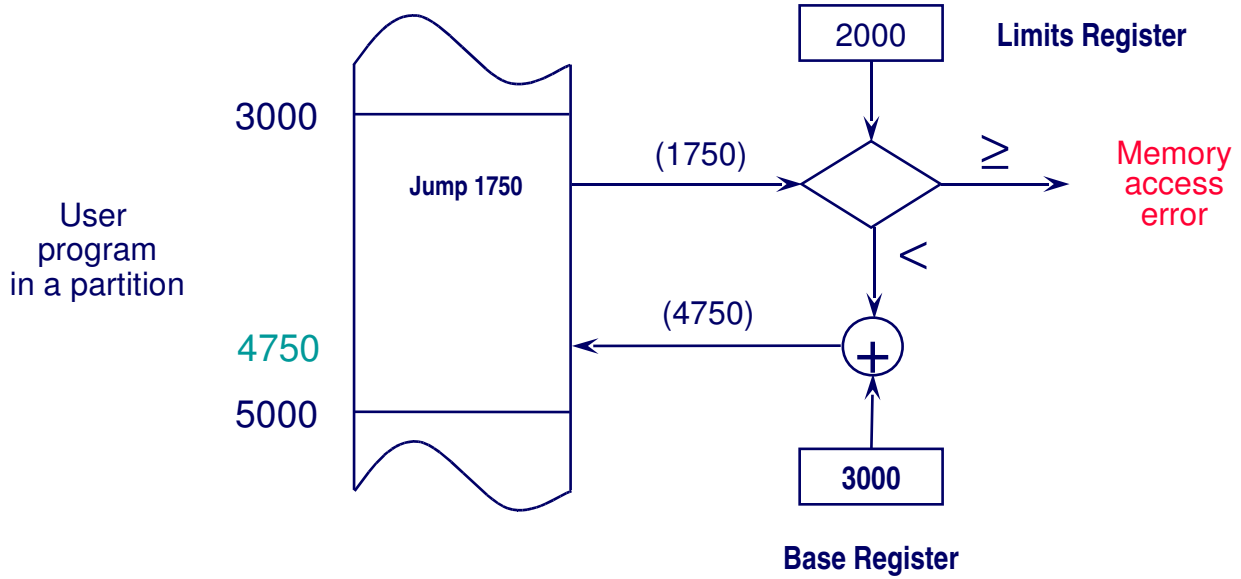
Partition allocation at different times

July 99

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Address translation

In order to provide basic protection among programs sharing the memory, both of the above partitioning techniques use a hardware capability known as *memory address mapping*, or address translation. In its simplest form, this mapping works as follows:



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Fragmentation

Fragmentation refers to the unused memory that the memory management system cannot allocate.

- *Internal fragmentation*

  Waste of memory *within* a partition, caused by the difference between the size of a partition and the process loaded. Severe in static partitioning schemes.

- *External fragmentation*

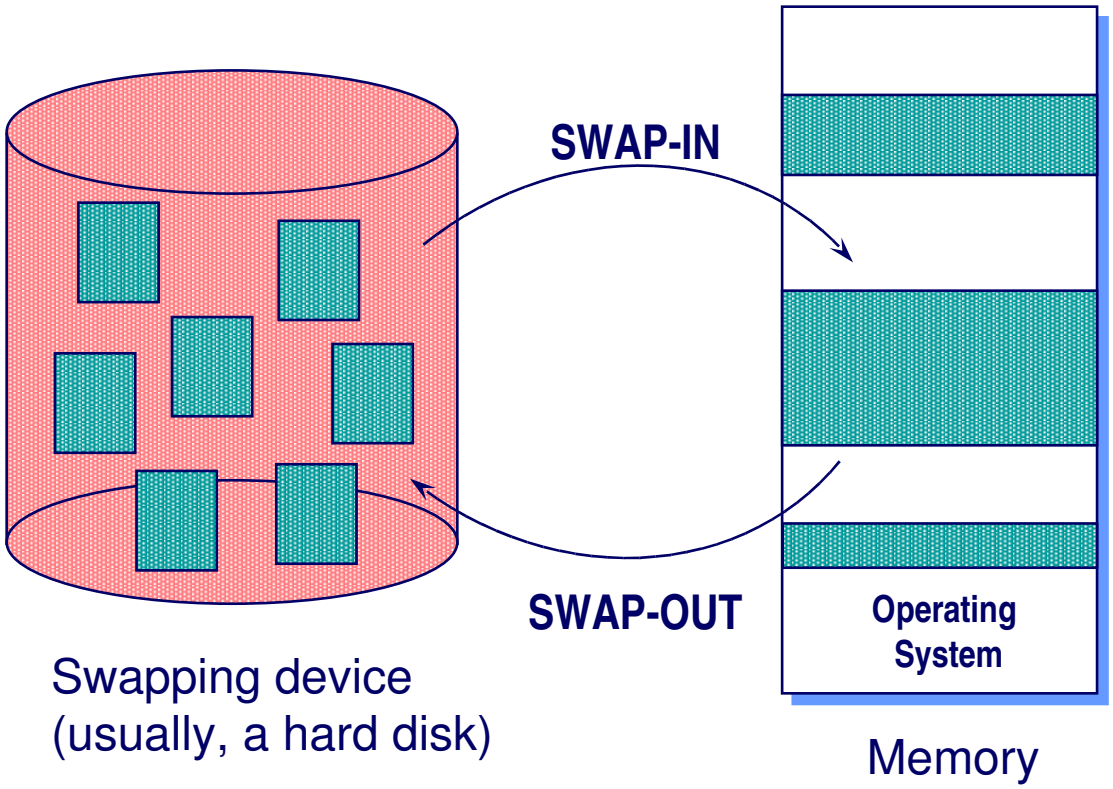
  Waste of memory *between* partitions, caused by scattered noncontiguous free space. Severe in dynamic partitioning schemes.

Compaction (aka relocation) is a technique that is used to overcome external fragmentation.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Swapping

The basic idea of swapping is to treat main memory as a "*preemptable*" resource. A high-speed swapping device is used as the backing storage of the preempted processes.



July 99

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Swapping *cont.*

Swapping is a <u>medium-term scheduling</u> method.

Memory becomes preemptable.

Swapping brings flexibility even to systems with fixed partitions, because:
"if needed, the operating system can always make room for high-priority jobs, no matter what!"

Note that, although the mechanics of swapping are fairly simple in principle, its implementation requires specific support (e.g., special file system and dynamic relocation) from the OS that uses it.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

More on swapping

The responsibilities of a swapper include:

- Selection of processes to swap out

  criteria: suspended/blocked state, low priority, time spent in memory

- Selection of processes to swap in

  criteria: time spent on swapping device, priority

- Allocation and management of swap space on a swapping device. Swap space can be:
  - system wide
  - dedicated (e.g., swap partition or disk)

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Memory protection

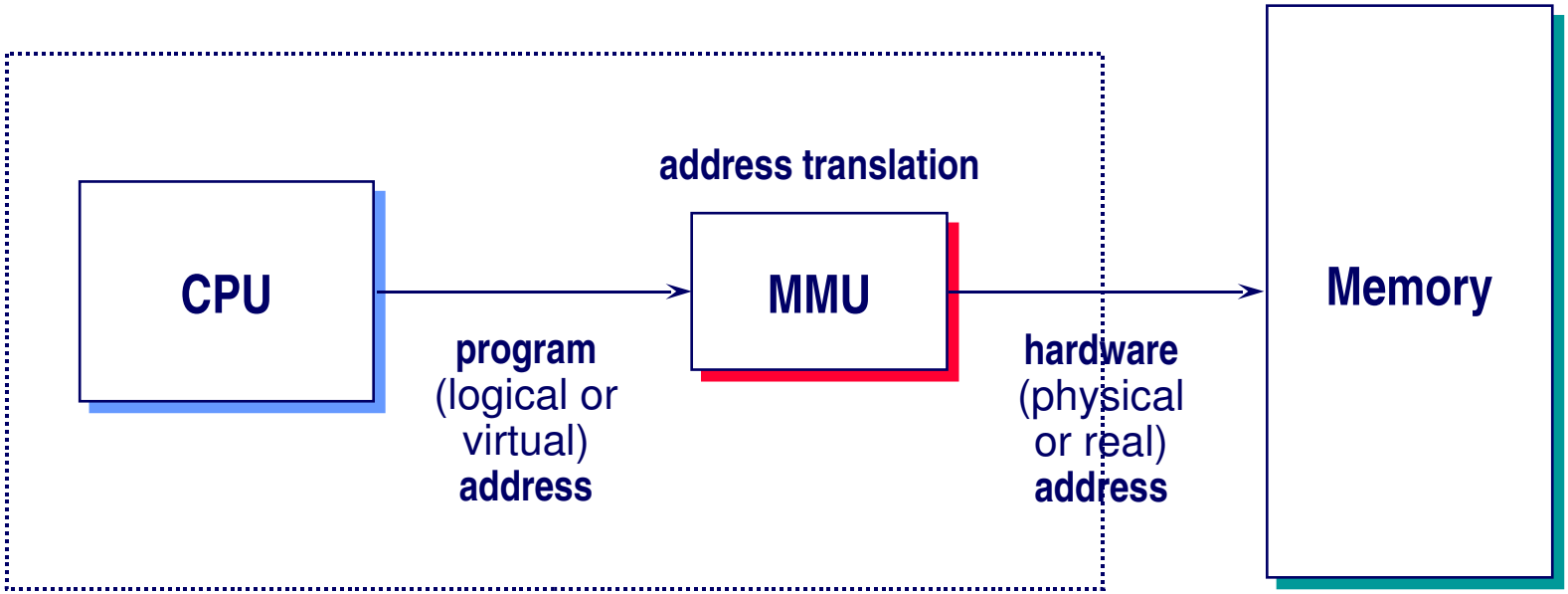
The second fundamental task of a memory management system is to *protect* programs *sharing* the memory from each other. This protection also covers the operating system itself. Memory protection can be provided at either of the two levels:

- Hardware:
  - *address translation* (most common!)
- Software:
  - language dependent: *strong typing*
  - language independent: *software fault isolation*

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Dynamic relocation

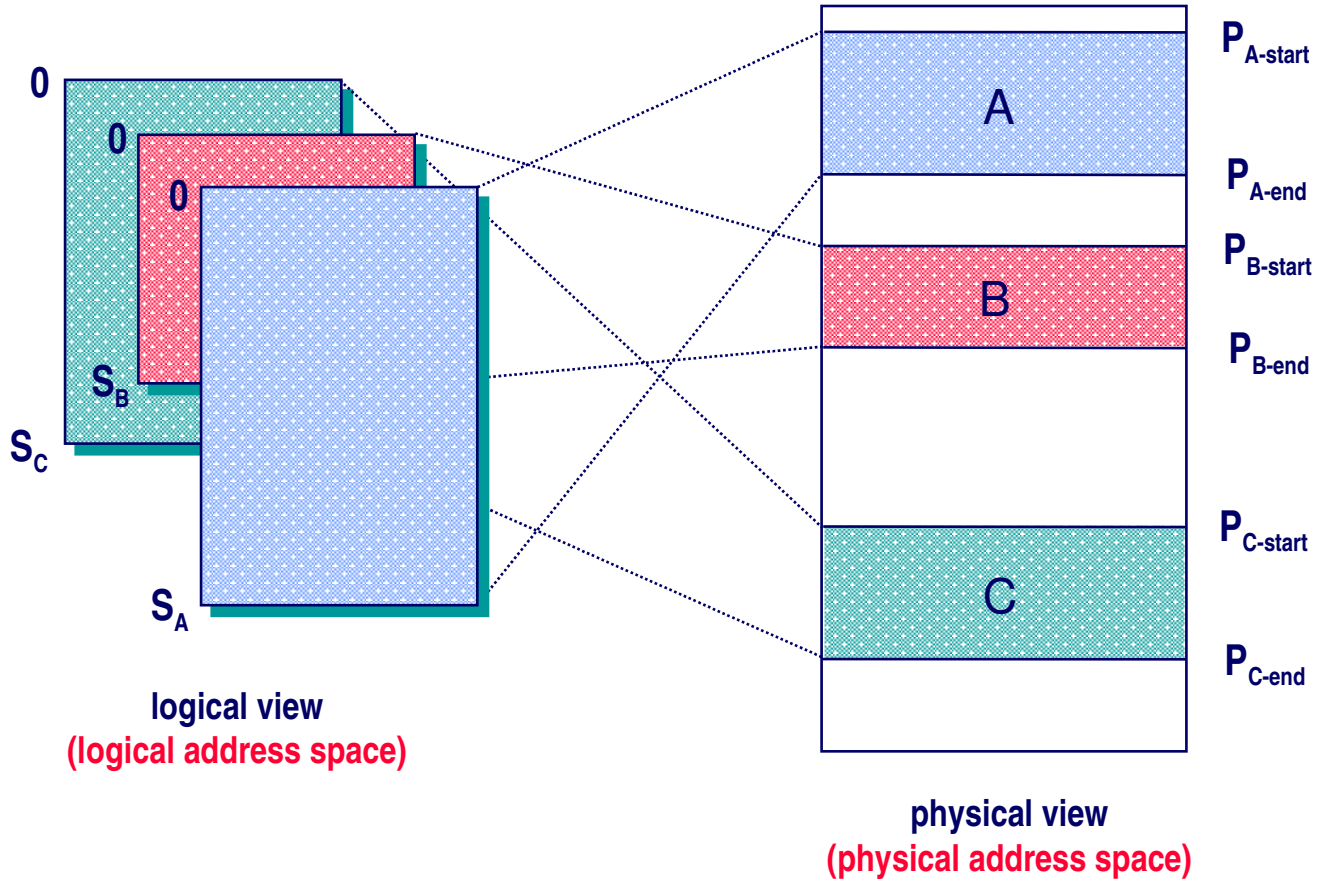
With *dynamic relocation*, each program-generated address (*logical address*) is translated to hardware address (*physical address*) at runtime for *every* reference, by a hardware device known as the memory management unit (MMU).



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Two views of memory

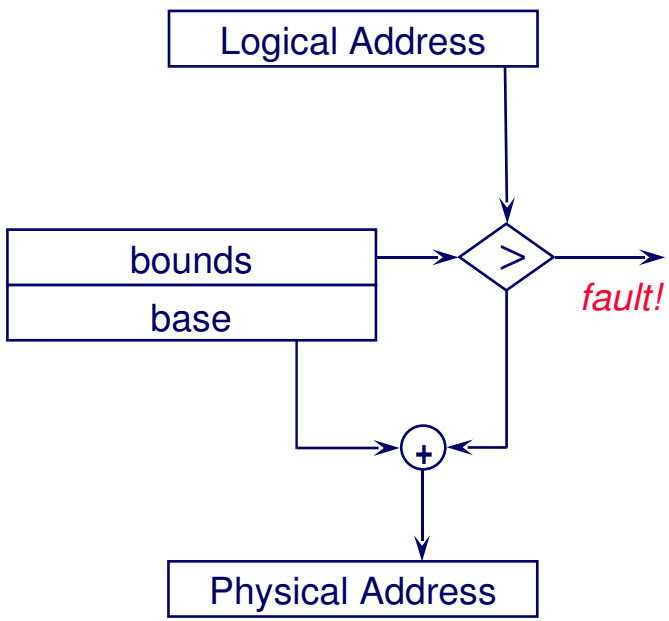
Dynamic relocation leads to two different views of main memory, called *address spaces*.



July 99

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Base and bounds relocation



In principle, this is the same technique used earlier by IBM 360 mainframes. Each program is loaded into a <u>contiguous</u> region of memory. This region appears to be "private" and the bounds register limits the range of the logical address of each program.

Hardware implementation is cheap and efficient: 2 registers plus an adder and a comparator.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Segmentation

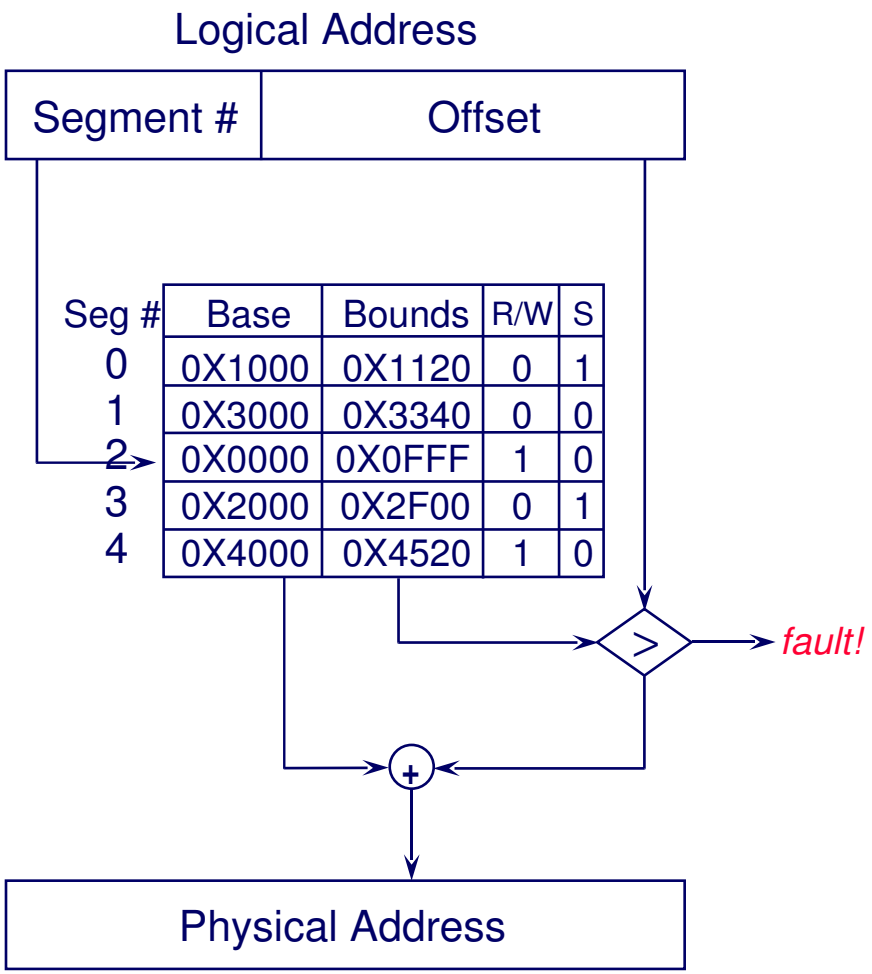
A segment is a region of contiguous memory. The most important problem with *base-and-bounds* relocation is that there is only one segment for each process.

Segmentation generalizes the base-and-bounds technique by allowing each process to be split over several segments (i.e., multiple base-bounds pairs). A segment table holds the base and bounds of each segment. Although the segments may be scattered in memory, each segment is mapped to a contiguous region.

Additional fields (Read/Write and Shared) in the segment table adds protection and sharing capabilities to segments (i.e., permissions)

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

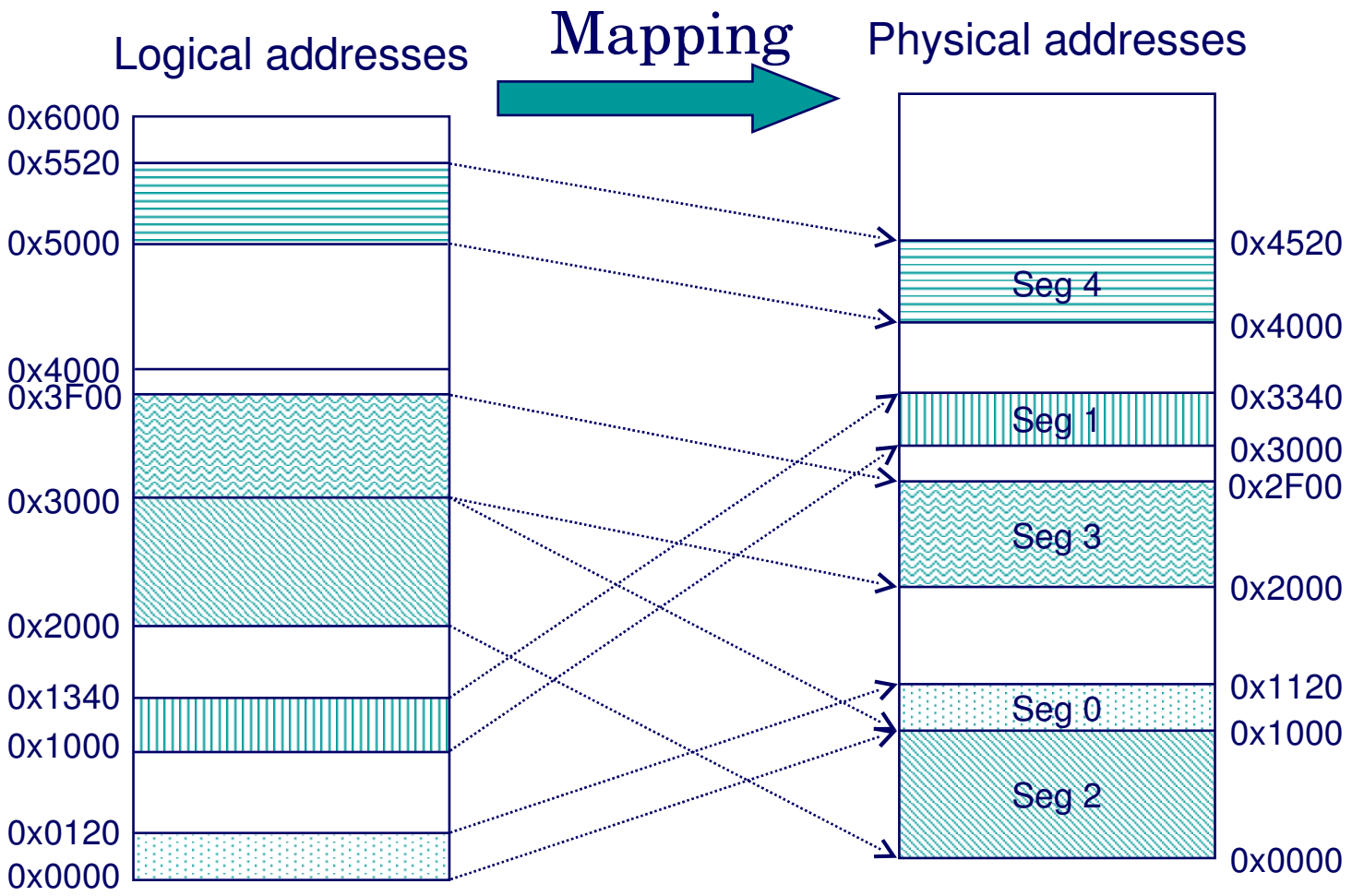
Relocation with segmentation



See next slide for memory allocation example.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Segmentation—*an example*



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

More on segments

When a process is created, an empty segment table is inserted into the process control block. Table entries are filled as new segments are allocated for the process.

The segments are returned to the free segment pool when the process terminates.

Segmentation, as well as the base and bounds approach, causes external fragmentation (because they require contiguous physical memory) and requires memory compaction.

An advantage of the approach is that only a segment, instead of a whole process, may be swapped to make room for the (new) process.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Paging

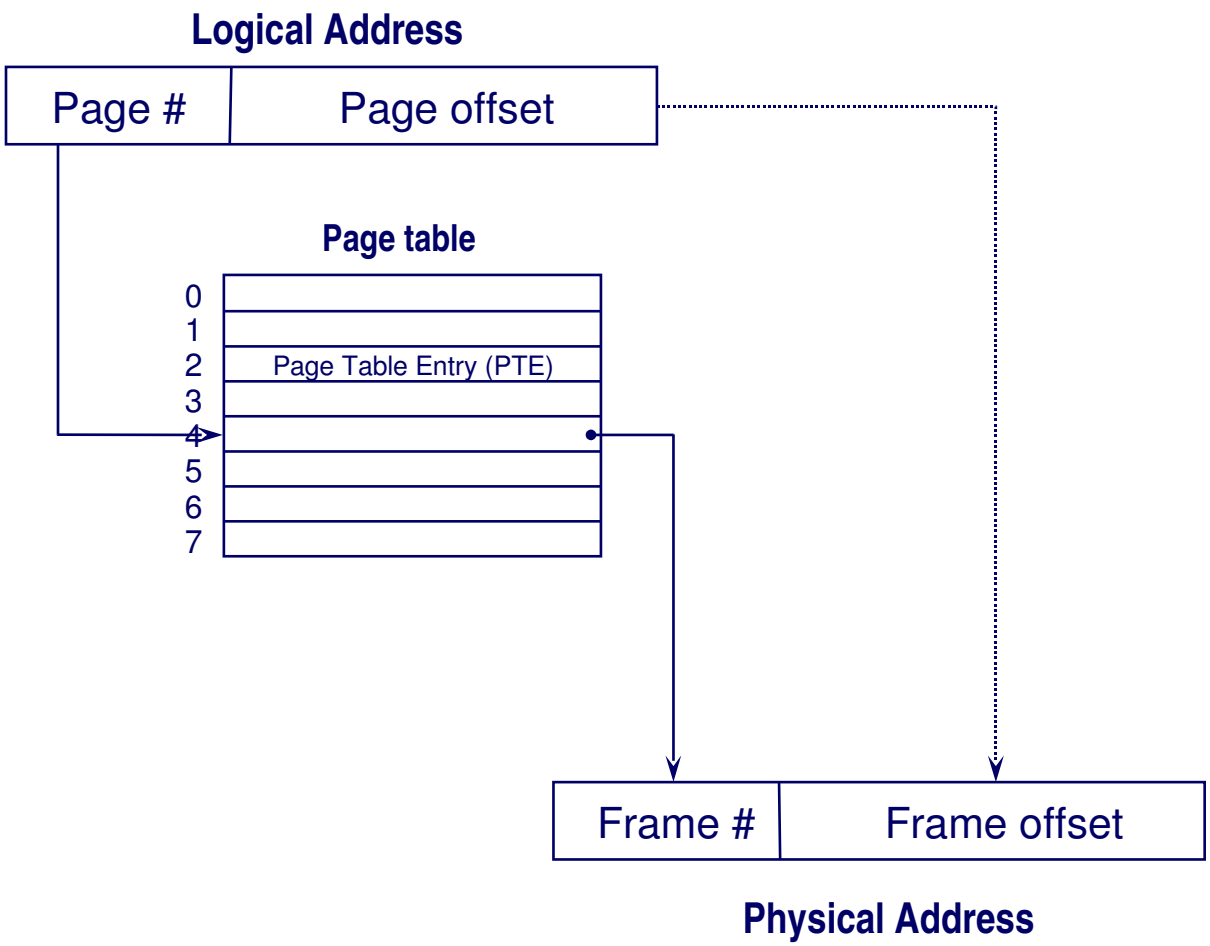
Physical memory is divided into a number of fixed size blocks, called *frames*. The logical memory is also divided into chunks of the same size, called *pages*. The size of frame/page is determined by the hardware and can be any value between 512 bytes (VAX) and 16 megabytes (MIPS 10000)! *Why have such large frames? Why not?*

A *page table* defines (maps) the base address of pages for each frame in the main memory.

The major goals of paging are to make memory allocation and swapping easier and to reduce fragmentation. Paging also allows allocation of non-contiguous memory (i.e., pages need not be adjacent.)

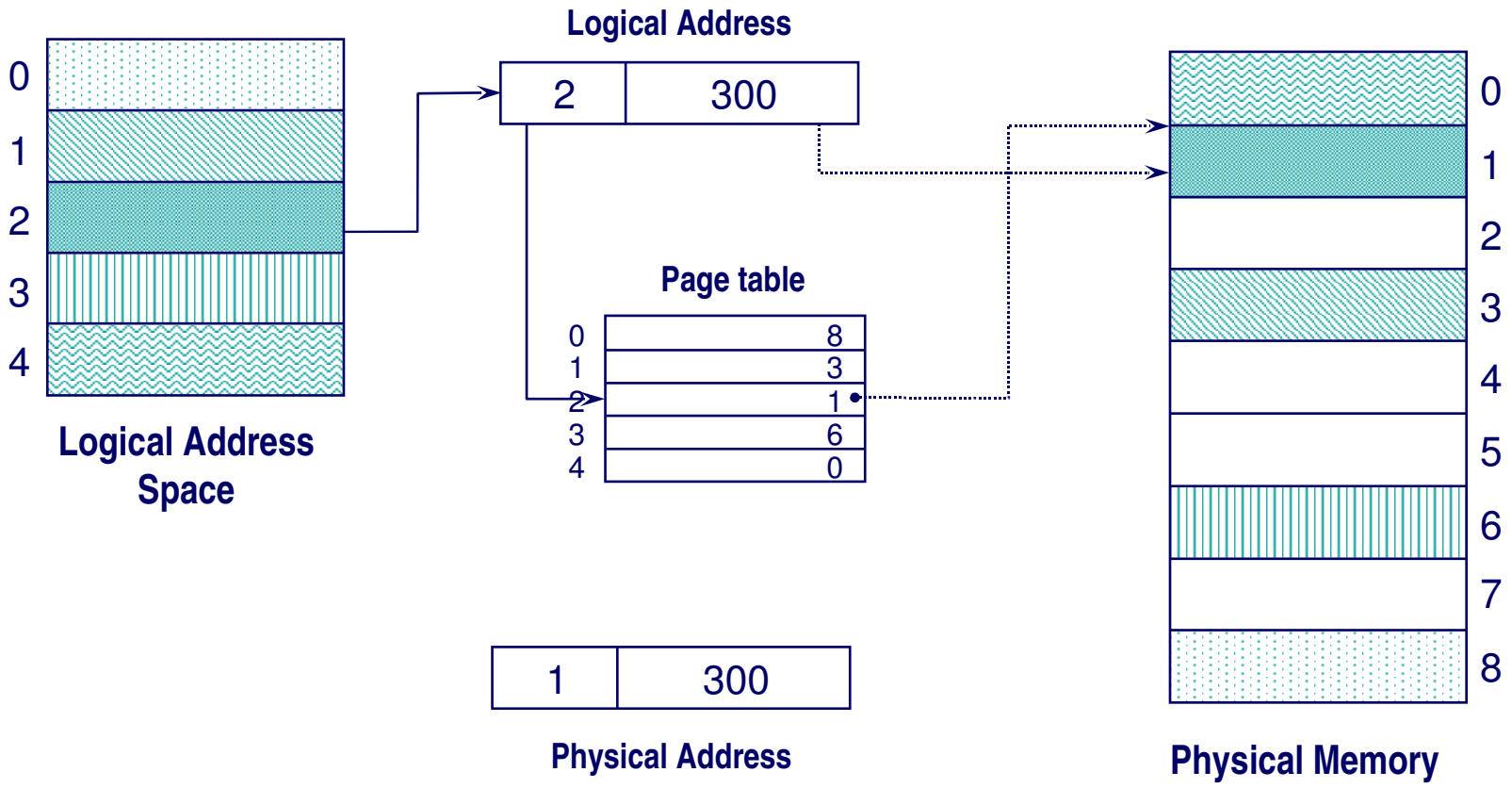
Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Relocation with paging



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Paging—*an example*



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Hardware support for paging

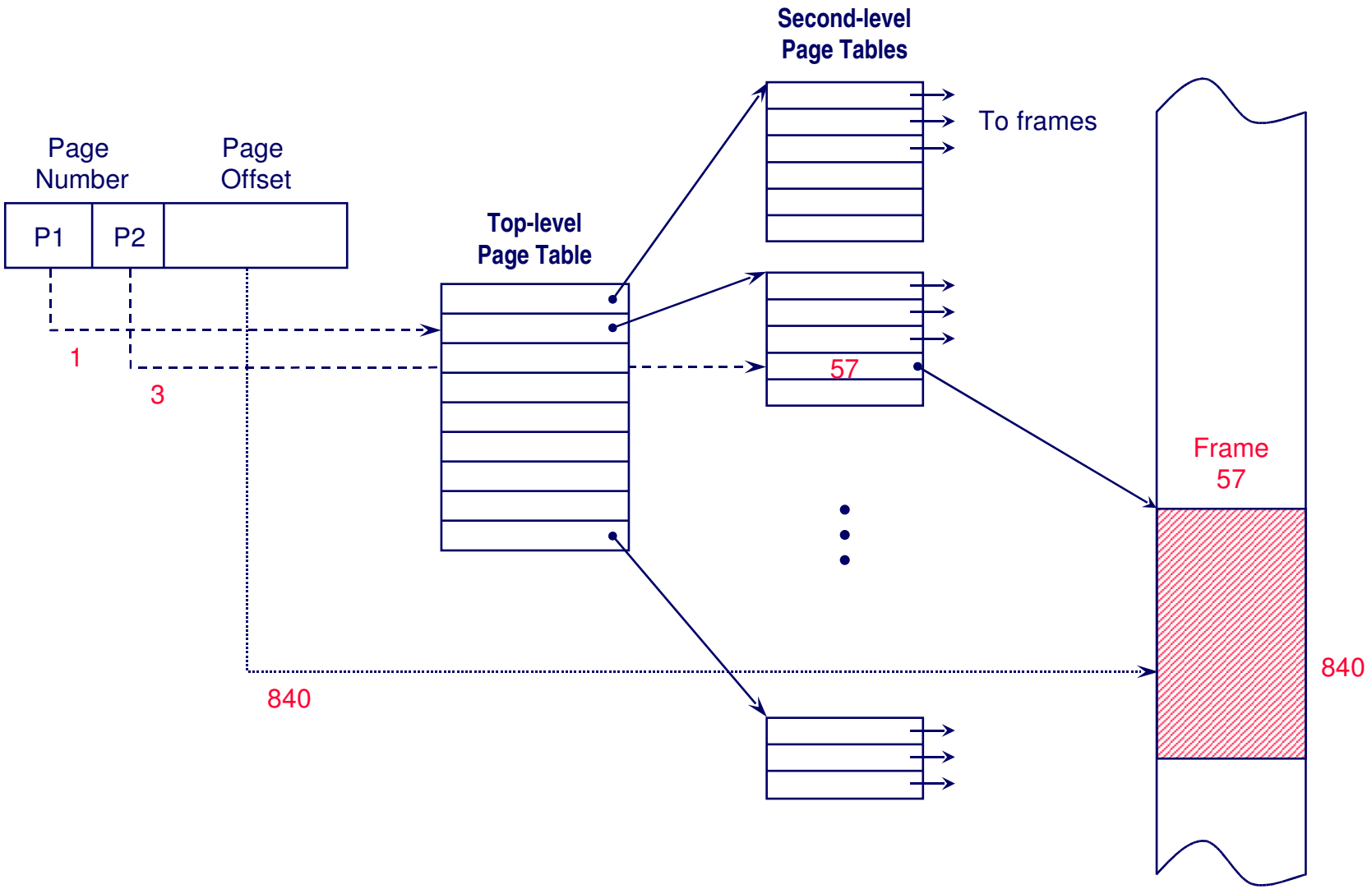
There are different hardware implementations of page tables to support paging.

- A set of dedicated registers, holding base addresses of frames.
- In memory page table with a *page table base register* (PTBR).
- Same as above with multi-level page tables.

With the latter two approaches, there is a constant overhead of accessing a memory location (*What? Why?*)

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

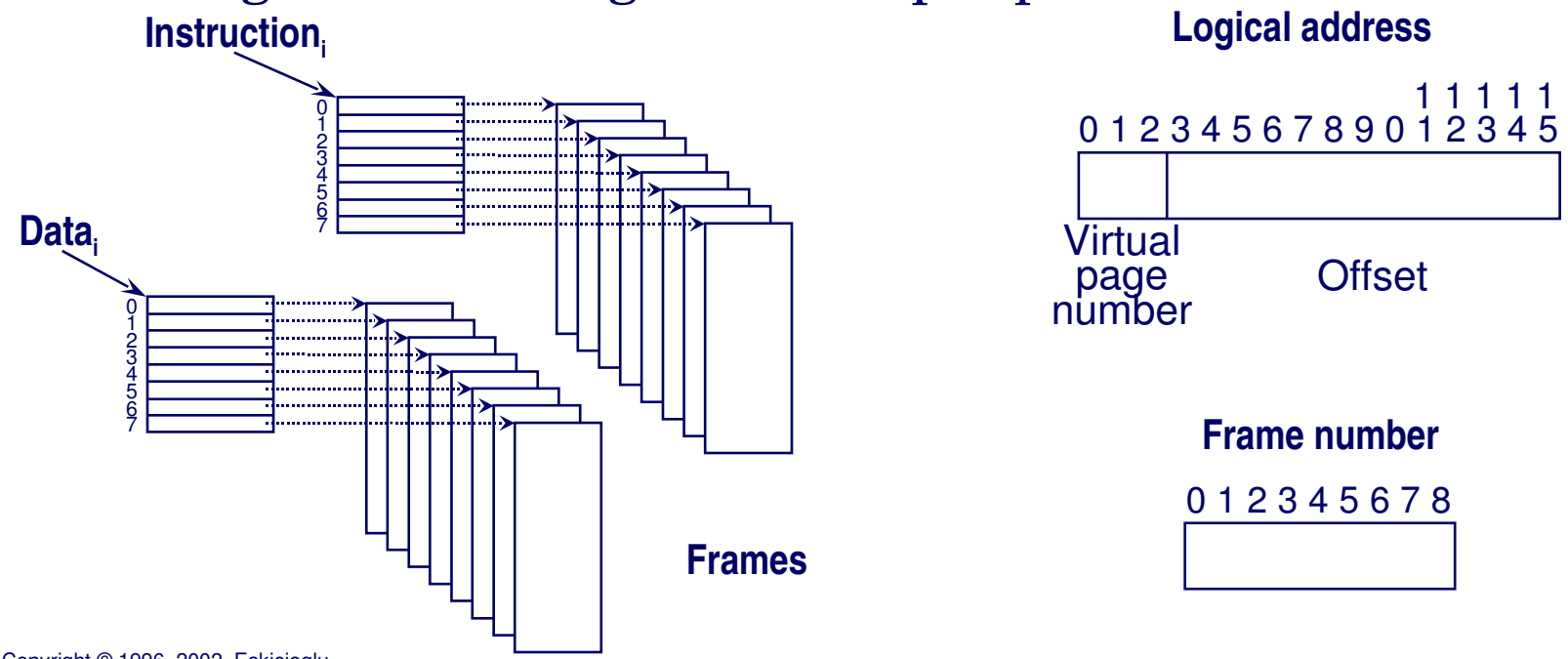
Multi-level paging—*an example*



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

One level paging—the PDP-11

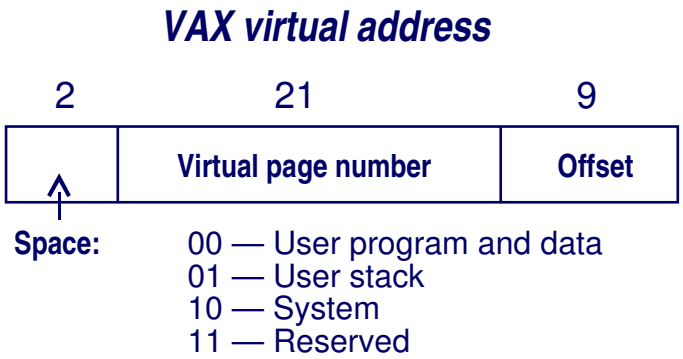
The larger PDP-11 models have 16-bit logical addresses (i.e., 64 KB) and up to 4MB of memory with page size of 8 KB. *Note: Physical address space larger than logical space!* There are two separate logical address spaces; one for instructions and one for data. The two page tables have eight entries, each controlling one of the eight frames per process.

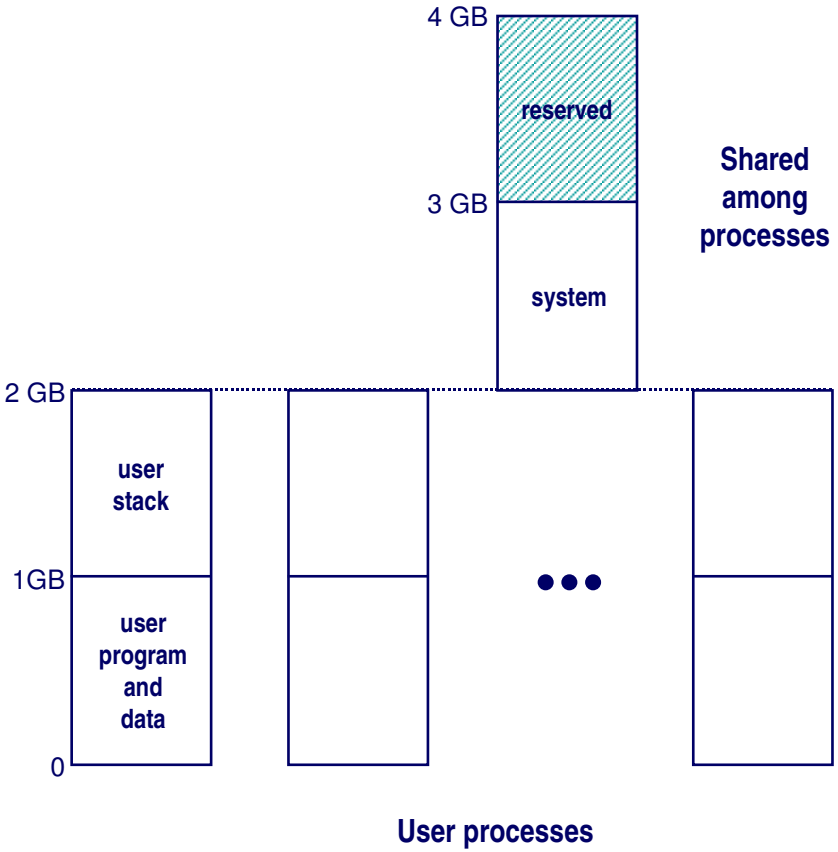


Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Two level paging—the VAX

The VAX is the successor of the PDP-11, with 32-bit logical addresses. The VAX has 512 byte pages.





Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Other MMU architectures

Some SPARC processors used by the Sun workstations have a paging MMU with three-level page tables and 4KB pages.

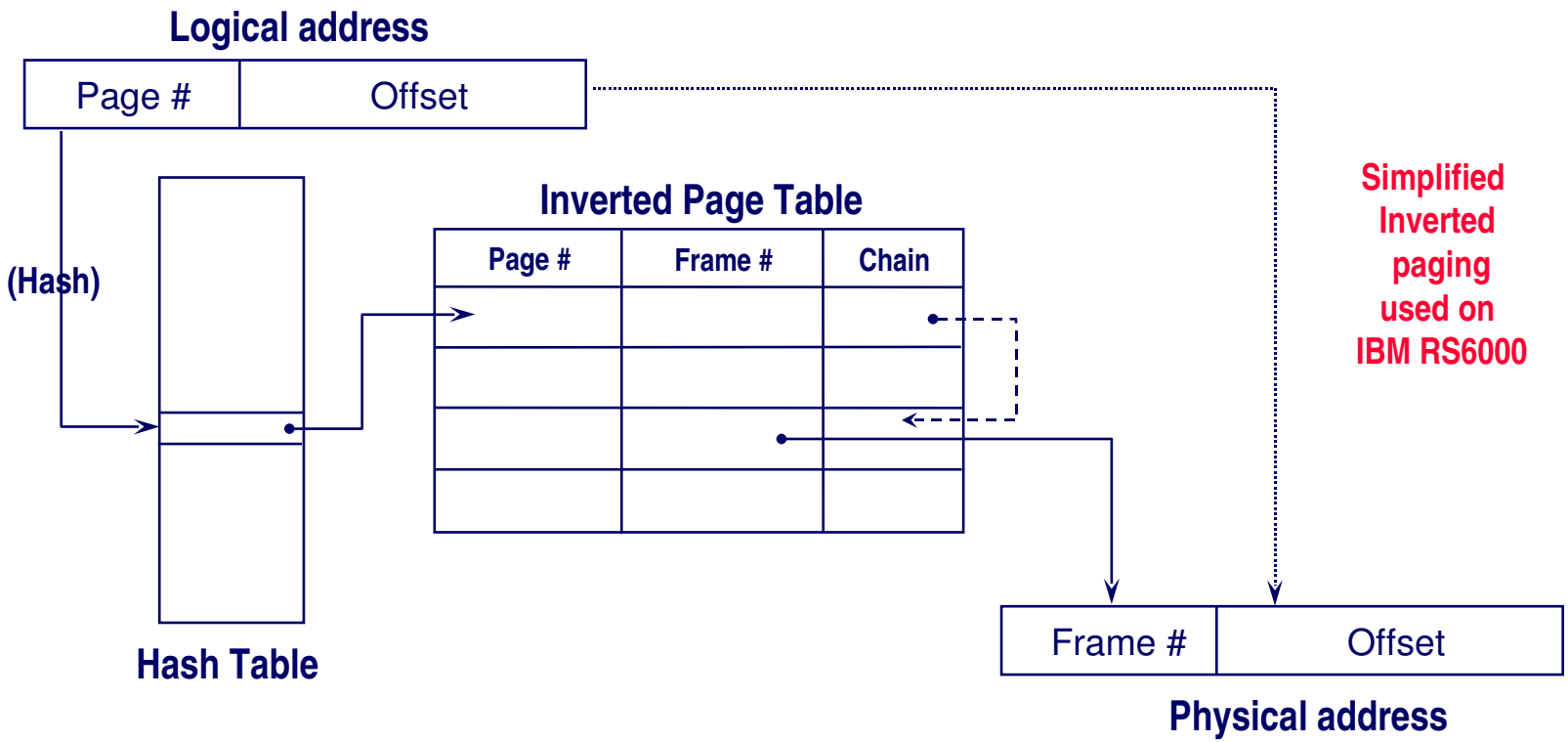
Motorola 68030 processor uses on-chip MMU with programmable multi-level (1 to 5) page tables and 256 byte to 32KB pages.

PowerPC processors support complex address translation mechanisms and, based on the implementation, provide 2^{80} (64-bit) and 2^{52} (32-bit) byte long logical address spaces.

Intel Pentium processors support both segmented and paged memory with 4KB pages.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

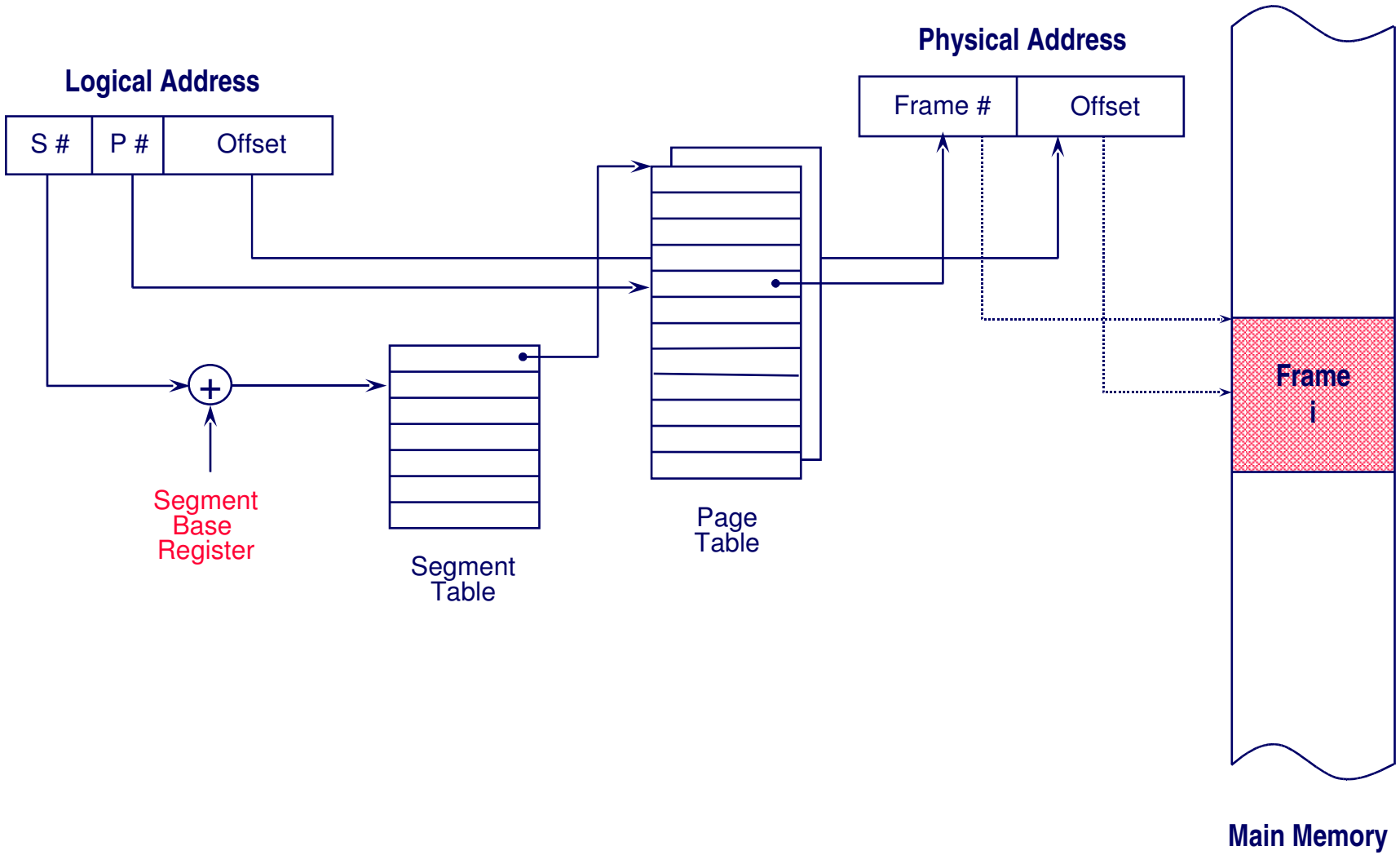
Inverted page tables



The inverted page table has <u>one entry for each memory frame</u>. Hashing is used to speedup table search. The inverted page table can either be per process or system-wide. In the latter case, a new entry, PID is added to the table. **Adv**: independent of size of address space; <u>small table(s) if we have large logical address spaces.</u>

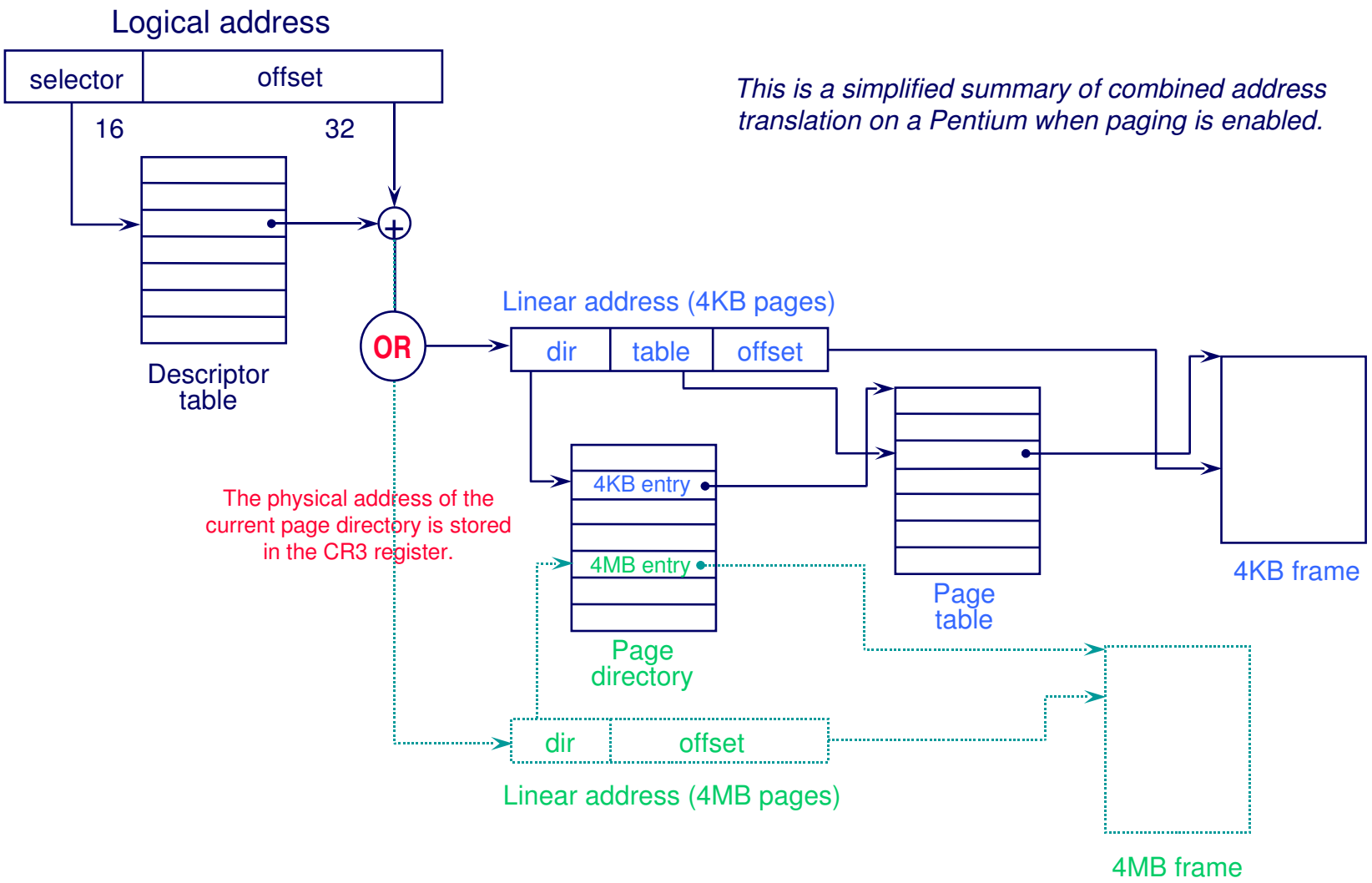
Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Segmentation with paging



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Address translation on a Pentium



This is a simplified summary of combined address translation on a Pentium when paging is enabled.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Associative memory

Problem: Both paging and segmentation schemes introduce extra memory references to access translation tables.
Solution? Translation buffers (like caches)

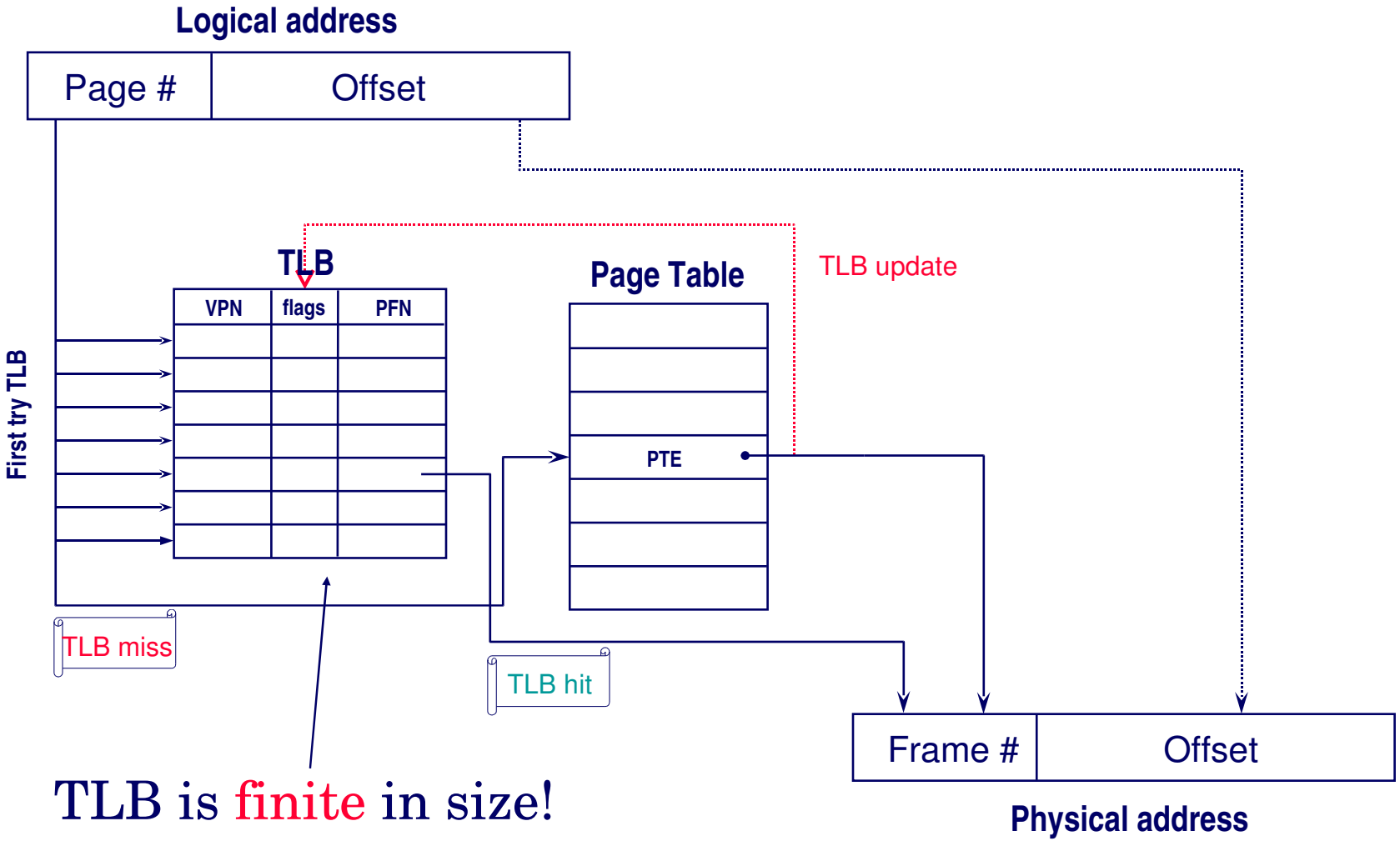
Based on the notion of *locality (at a given time a process is only using a few pages or segments)*, a very fast but small *associative* (content addressable) *memory* is used to store a few of the translation table entries. This memory is known as a *translation look-aside buffer* or *TLB*.

MIPS R2000/R3000 TLB entry (64 bits)

Virtual Page Number	PID	000000	Physical Frame Number	Flags	00000000
20	6	6	20	4	8

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

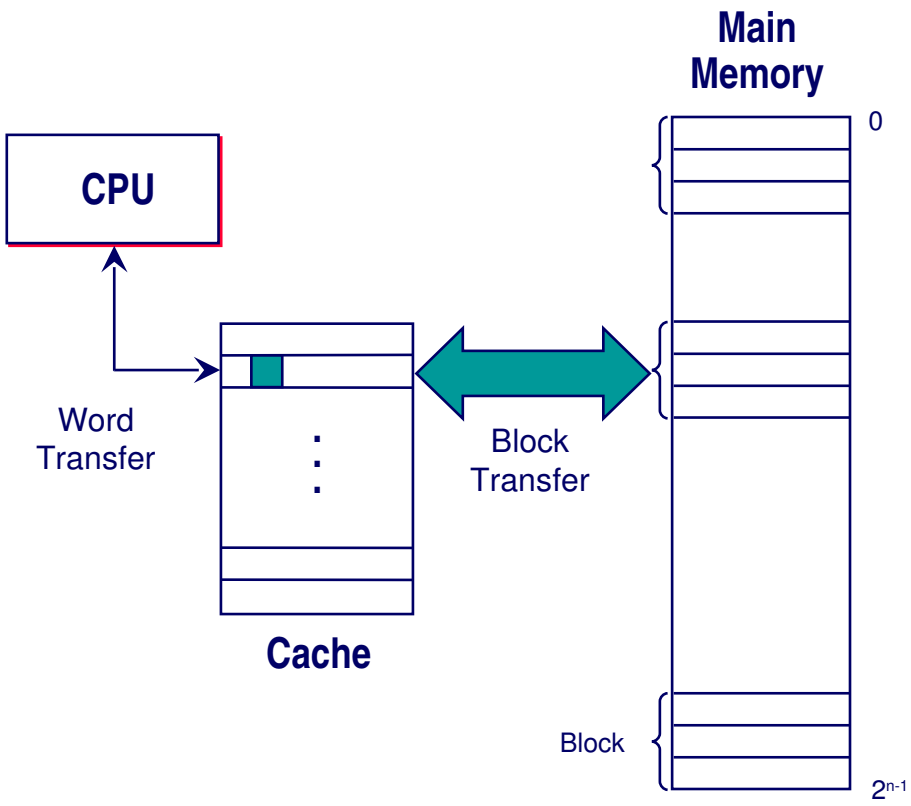
Address translation with TLB



TLB is finite in size!

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Memory caching



Similar to storing memory addresses in TLBs, frequently used data in main memory can also be stored in fast buffers, called *cache memory*, or simply *cache*. Basically, memory access occurs as follows:

```
for each memory reference
  if data is not in cache <miss>
     if cache is full
        remove some data
     if read access
        issue memory read
        place data in cache
        return data
  else <hit>
     if read access
        return data
     else
        store data in cache
```

The idea is to make *frequent* memory accesses faster!

July 99

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Cache terminology

Cache hit: item is in the cache.

Cache miss: item is not in the cache; must do a full operation.

Categories of cache miss:

- *Compulsory*: the first reference will always miss.
- *Capacity*: non-compulsory misses because of limited cache size.

Effective access time:

P(hit) * cost of hit + P(miss)* cost of miss, where
P(hit) = 1 - P(miss)

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Issues in cache design

Although there are many different cache designs, all share a few common design elements:

- **Cache size**—how big is the cache?

  The cache only contains a copy of portions of main memory. The larger the cache the slower it is. Common sizes vary between *4KB* and *4MB*.

- **Mapping function**—how to map main memory blocks into cache lines?

  Common schemes are: *direct, fully associative, and set associative.*

- **Replacement algorithm**—which line will be evicted if the cache lines are full and a new block of memory is needed.

  A replacement algorithm, such as *LRU, FIFO, LFU, or Random* is needed only for *associative* mapping (Why?)

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Issues in cache design *cont.*

- **Write policy**—What if CPU modifies a (cached) location?

  This design issue deals with store operations to cached memory locations. Two basic approaches are: *write through* (modify the original memory location as well as the cached data) *and write back* (update the memory location only when the cache line is evicted.)

- **Block (or line) size**—how many words can each line hold?

  Studies have shown that a cache line of 4 to 8 addressable units (bytes or words) provide close to optimal number of hits.

- **Number of caches**—how many levels? Unified or split cache for data and instructions?

  Studies have shown that a second level cache improves performance. Pentium and Power PC processors each have on-chip level-1 (L1) split caches. Pentium Pro processors have on-chip level-2 (L2) cache, as well.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

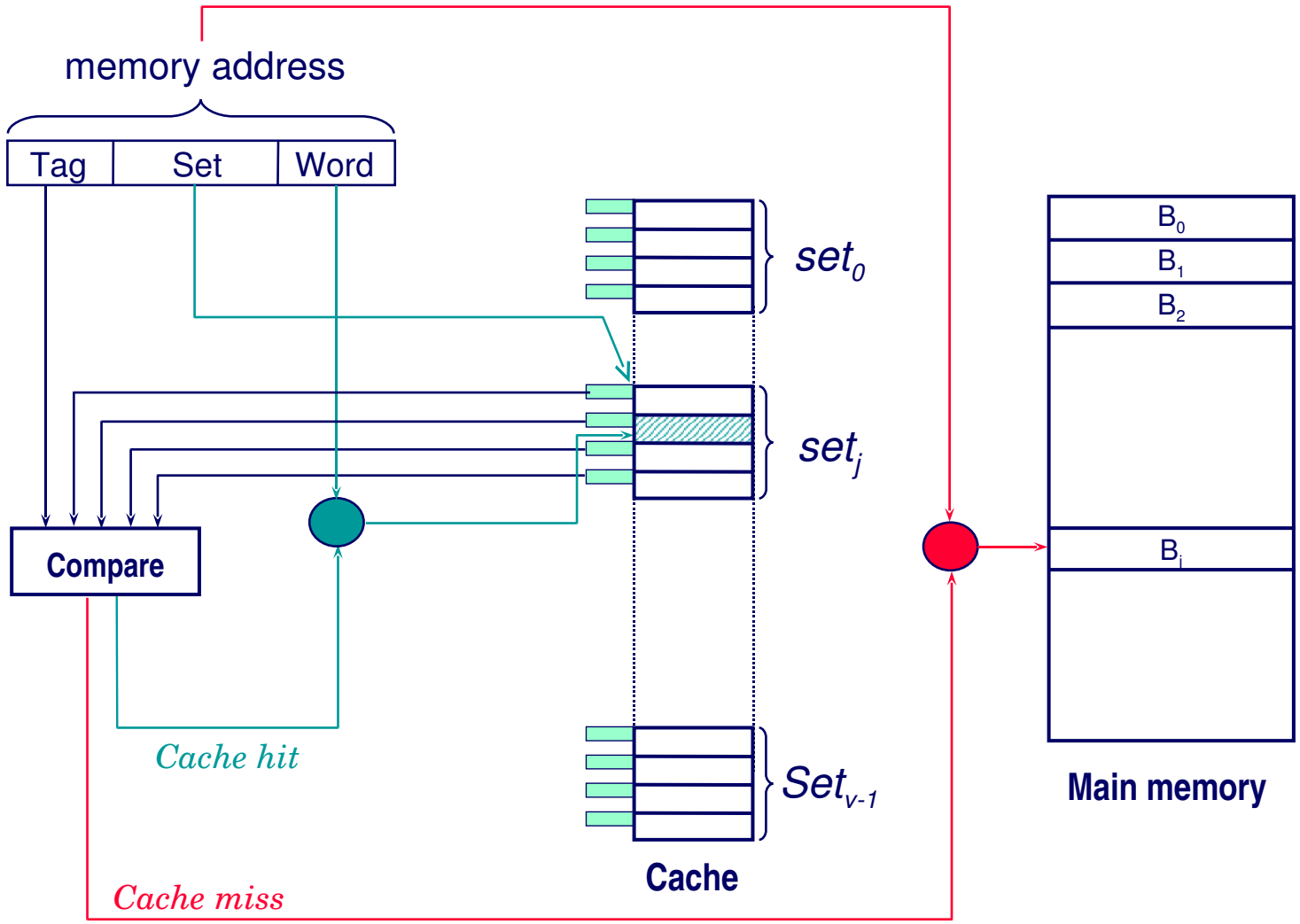
Mapping function

Since there are more main memory blocks ($Block_i$ *for i=0 to n*) than cache lines ($Line_j$ for *j=0 to m, and n >> m*), an algorithm is needed for mapping main memory blocks to cache lines.

- *Direct mapping*—maps each block of memory into only one possible cache line. $Block_i$ maps to $Line_j$, where $i = j$ *modulo m*.
- *Associative mapping*—maps any memory block to any line of the cache.
- *Set associative mapping*—cache lines are grouped into sets and a memory block can be mapped to any line of a cache set. $Block_i$ maps to Set_j where *i=j modulo v and v is the number of sets with k lines each*.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Set associative cache organization



Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Dynamic memory allocation

Static memory allocation schemes are not sufficient at run-time because of the unpredictable nature of executing programs. For certain programs, such as recursive functions or programs that use complex data structures, the memory needs cannot be known in advance.

Two basic operations are needed: *allocate* and *free*.

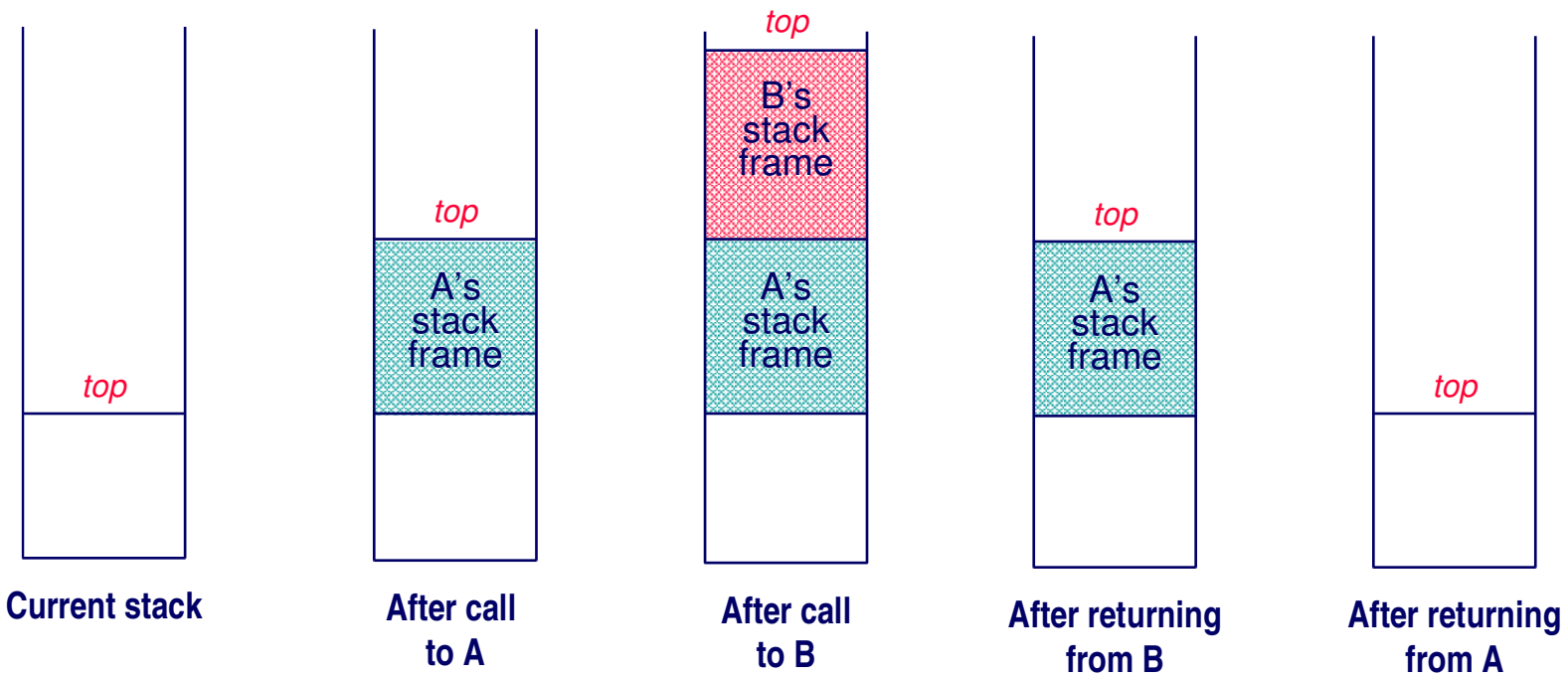
For example, in UNIX, these are `malloc()` and `free()`.

Dynamic allocation can be handled using either *stack* (hierarchical, restrictive) or *heap* (more general, but less efficient) allocation.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Stack organization

Memory allocation and freeing operations are partially predictable. Since the organization is hierarchical, the freeing operates in reverse (opposite) order.

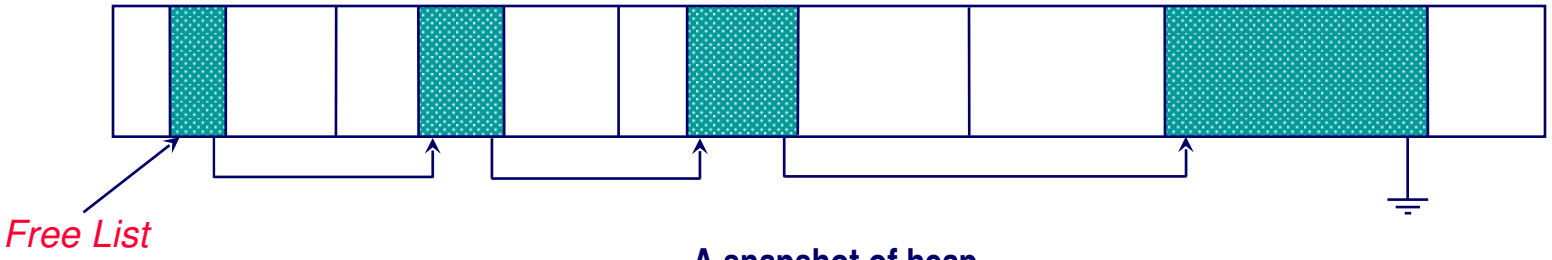


Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Heap organization

Allocation and release of heap space is totally random. Heaps are used for allocation of arbitrary list structures and complex data organizations. As programs execute (and allocate and free structures), heap space will fill with holes (unallocated space, i.e., fragmentation)

Statistically, for first-fit, "even with some optimization, given N allocated blocks, another $0.5N$ blocks will be lost due to fragmentation." This result is known as the *fifty percent rule*. (pg. 254 in Silberschatz textbook)



A snapshot of heap

July 99

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

“Free” memory management

- **Bit maps**

  This technique divides memory into fixed-size blocks (e.g., sectors of 256-byte blocks) and keeps an array of bits (bit map), one bit for each block.

- **Linked lists**

  A *free list* keeps track of the unused memory. There are several algorithms that can be used, depending on the way the unused memory blocks are allocated: *first fit, best fit, and worst fit*.

- **Buddy system**

  This algorithm takes advantage of binary systems. As a process requests memory, it is given a block of memory with a size of power two in which the process can just fit.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Reclaiming memory

How do we know *when* memory can be freed?

It is trivial when a memory block is used by one process. However, this task becomes difficult when a block is shared (e.g., accessed through pointers) by several processes.

Two problems with reclaiming memory:

- **Dangling pointers**: occur when the original allocator *frees* a shared pointer.
- **Memory leaks**: occur when we forget to free storage, even when it will not or cannot be used again. This is a common and serious problem; lots of tools. Unacceptable in an OS.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Reference counts

Memory reclamation can be done by keeping track of reference counters (i.e., the outstanding pointers to each block of memory.) When the counter goes down to zero, the memory block is freed. This scheme works fine with hierarchical structures, but becomes tricky with circular structures.

Examples:

- Smalltalk and Java use a similar scheme.
- UNIX's in-memory file descriptors
- And, after a system crash, `fsck` program runs (during rebooting) to check the integrity of on-disk file systems.

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)

Garbage collection

As an alternative to an explicit free operation, some systems implicitly do so (free storage) by simply deleting pointers. These systems search through all deleted pointers and reclaim the storage referenced by them.

Some languages, e.g., Lisp and Java, support this kind of “reclaimed” (free) memory management.

Garbage collection is often expensive; it could use more than 20% of the total CPU time! Also, it is difficult to code and debug garbage collectors.

But, garbage collection eliminates a large class of application programmer errors (a good thing)!

Copyright © 1996–2002 Eskicioglu and Marsland (and Prentice-Hall and Paul Lu)